```
  shutdown: graceful                    ❶
  tomcat:
    connection-timeout: 2s
    keep-alive-timeout: 15s
    threads:
      max: 50
      min-spare: 5

spring:
  application:
    name: catalog-service
  lifecycle:
    timeout-per-shutdown-phase: 15s     ❷
...
```

❶ Enable graceful shutdown

❷ Defines a 15 s grace period

Since we've modified the application source code, we need to build a new container image and load it into minikube. That's not very efficient, is it? Later in the chapter, I'll show you a better way. For now, follow the procedure I described earlier to package Catalog Service as a container image (`./gradlew bootBuildImage`) and load it into the Kubernetes cluster we are using for Polar Bookshop (`minikube image load catalog -service --profile polar`).

After enabling application support for graceful shutdown, you need to update the Deployment manifest accordingly. When a Pod has to be terminated (for example, during a downscaling process or as part of an upgrade), Kubernetes sends a `SIGTERM` signal to it. Spring Boot will intercept that signal and start shutting down gracefully. By default, Kubernetes waits for a grace period of 30 seconds. If the Pod is not terminated after that period, Kubernetes sends a `SIGKILL` signal to force the Pod's termination. Since the Spring Boot grace period is lower than the Kubernetes one, the application is in control of when it will terminate.